Домашнее задание 4 по курсу «BI-аналитика и визуализация данных»

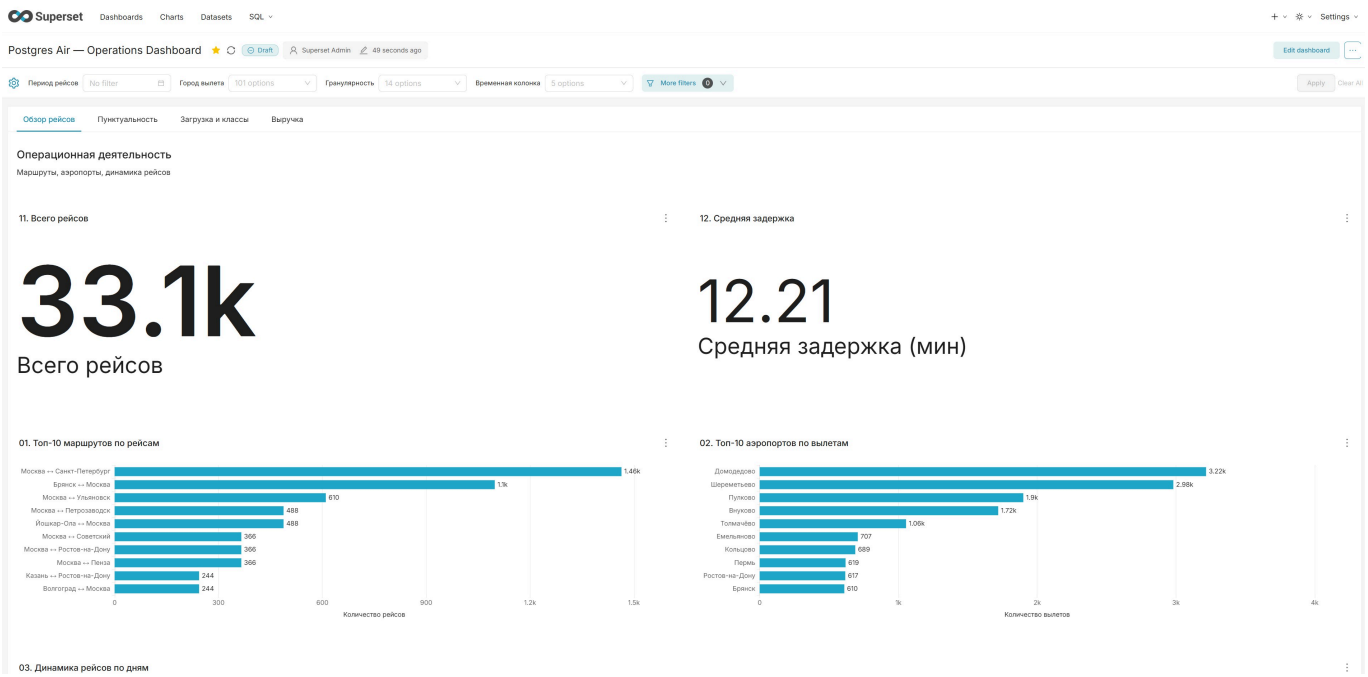
Выполнил - Третьяков Александр Юрьевич

Решение

Блок 1

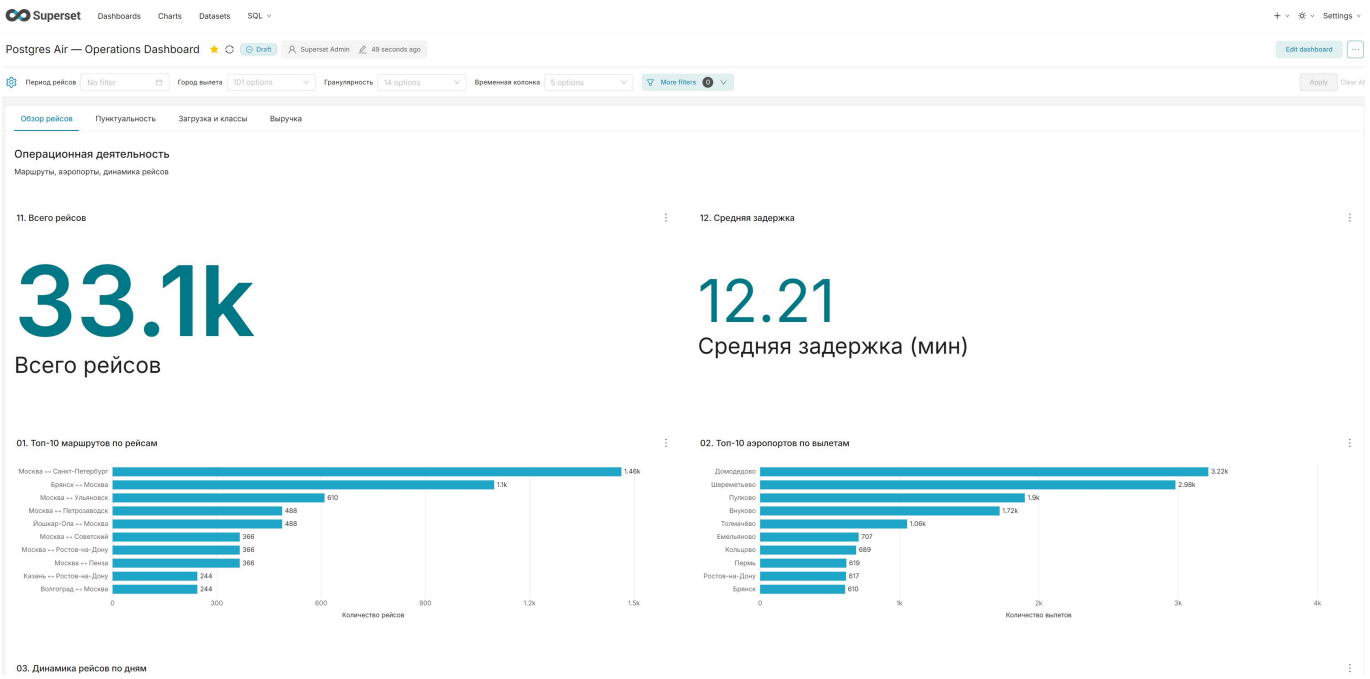
1) Расположите рядом на дашборде 2 визуализации вида Big Number, обратитесь к одной из них по ID и добавьте заметную кастомизацию с помощью CSS на свой вкус.

```
/* Пункт 1 — по ID */
#chart-id-126 .header-line {
    font-size: 180px !important;
    font-weight: 700 !important;
}
```



2) Обратитесь к классу визуализаций Big Number и добавьте заметную кастомизацию с помощью CSS на свой вкус.

```
.big_number_total .header-line {
    color: #007A87  !important;
}
```

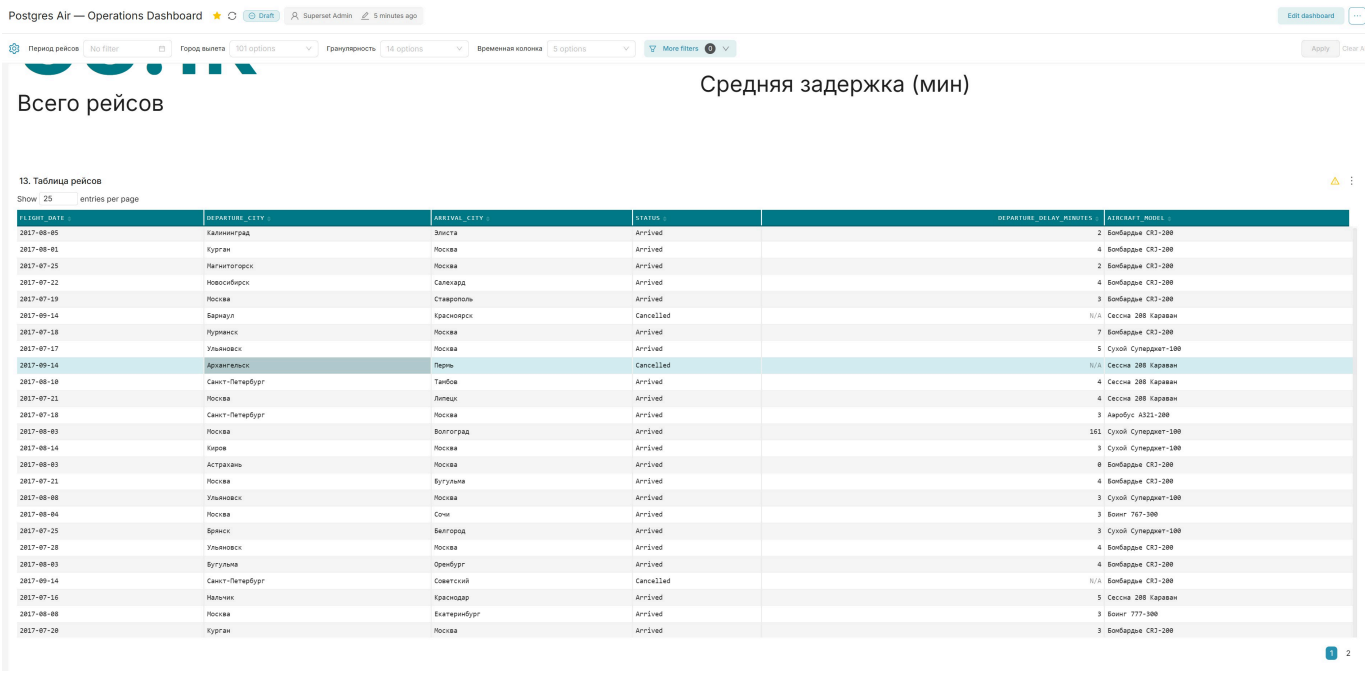


3) Добавьте (при необходимости) таблицу и кастомизируйте ее с помощью CSS на свой вкус.

```
.dashboard-chart-id-128 .table {
    font-family: 'Consolas', 'Courier New', monospace !important;
    font-size: 13px !important;
}

.dashboard-chart-id-128 .table th {
    background: #007A87 !important;
    color: #ffffff !important;
    text-transform: uppercase !important;
    font-size: 12px !important;
    letter-spacing: 1px !important;
    border-bottom: 2px solid #005f6b !important;
}

.dashboard-chart-id-128 .table tbody tr:hover {
    background: #d4eef1 !important;
}
```

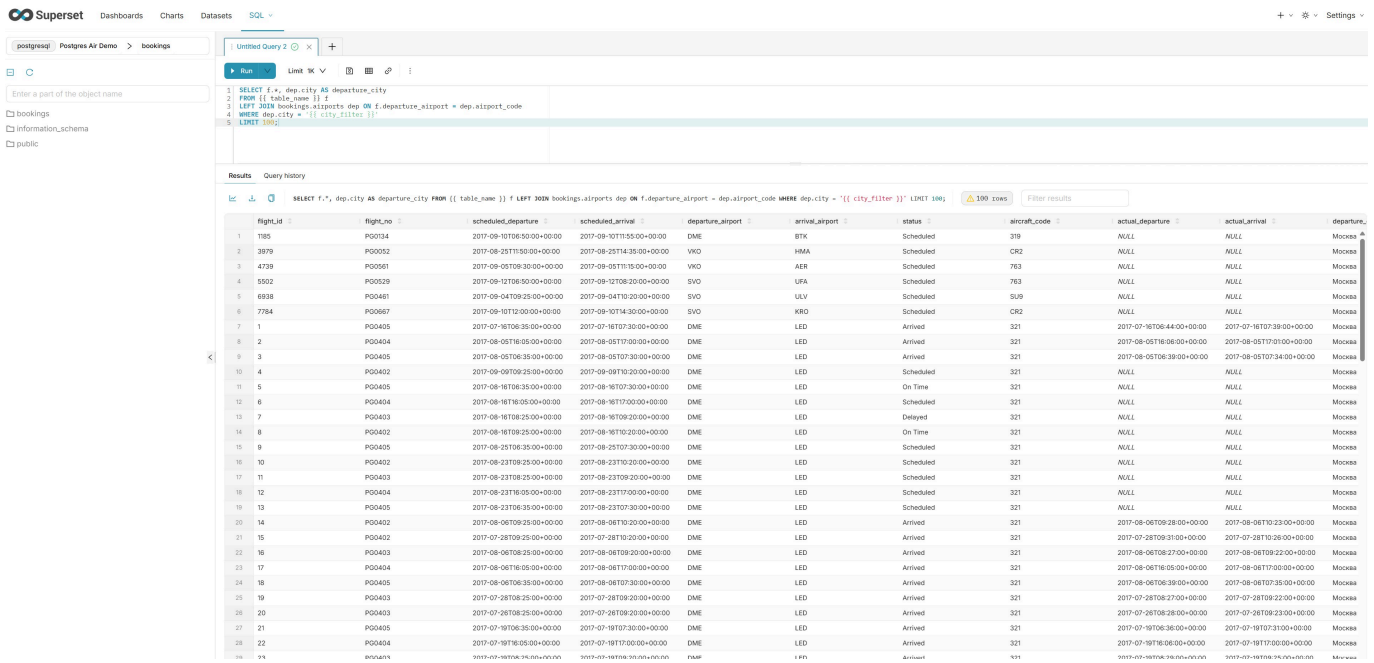


Блок 2

4) Возьмите любой успешный запрос с простым фильтром из ранее запущенных, в SQL Lab через в Parameters добавьте название таблицы и обратитесь к ней в запросе через шаблон Jinja. Значение фильтра поместите в переменную и добавьте его в запрос с помощью Jinja.

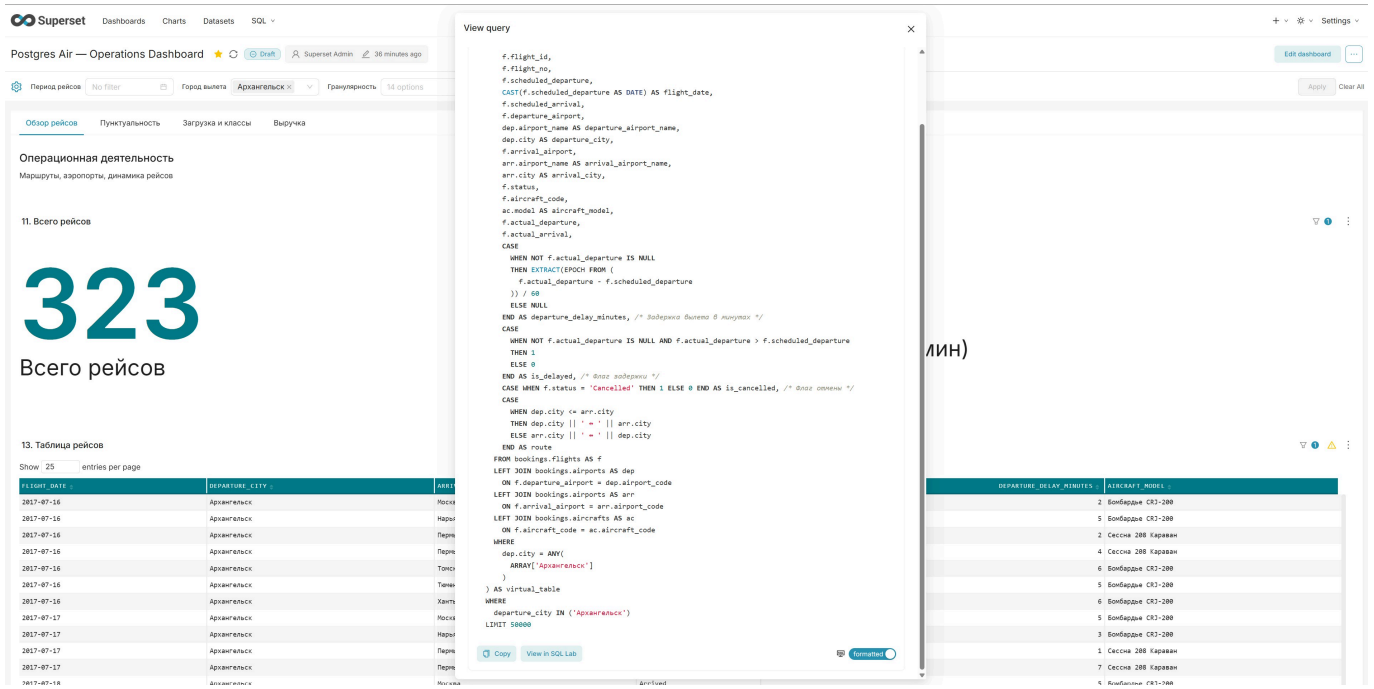
```
SELECT f.*, dep.city AS departure_city
FROM {{ table_name }} f
LEFT JOIN bookings.airports dep ON f.departure_airport = dep.airport_code
WHERE dep.city = '{{ city_filter }}'
LIMIT 100;
```

```
{
  "table_name": "bookings.flights",
  "city_filter": "Москва"
}
```



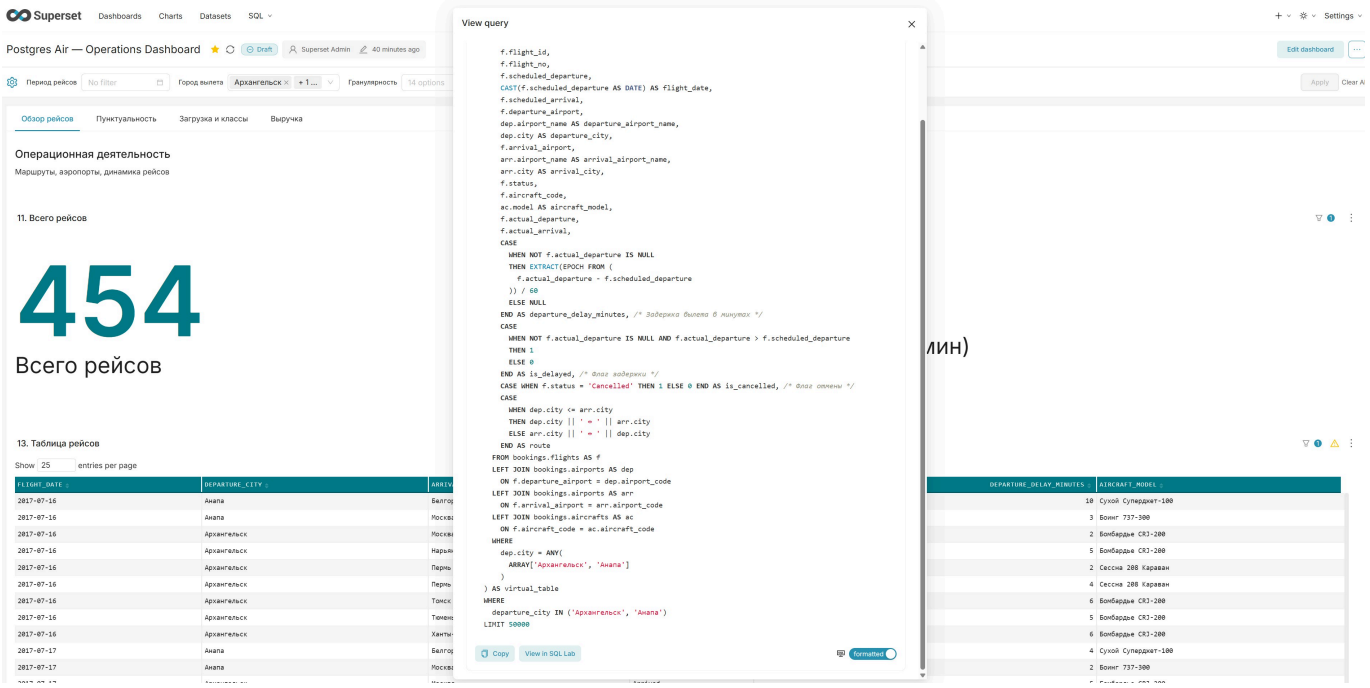
5) Далее используйте готовый дашборд (например, из ДЗ_3) и соответствующий виртуальный датасет. У вас есть обязательный фильтр без возможности множественного выбора. Добавьте в запрос Jinja-шаблон с минимально необходимым функционалом.

```
{% if filter_values('departure_city') %}
WHERE dep.city = ANY({{ filter_values('departure_city') }})
{% endif %}
```



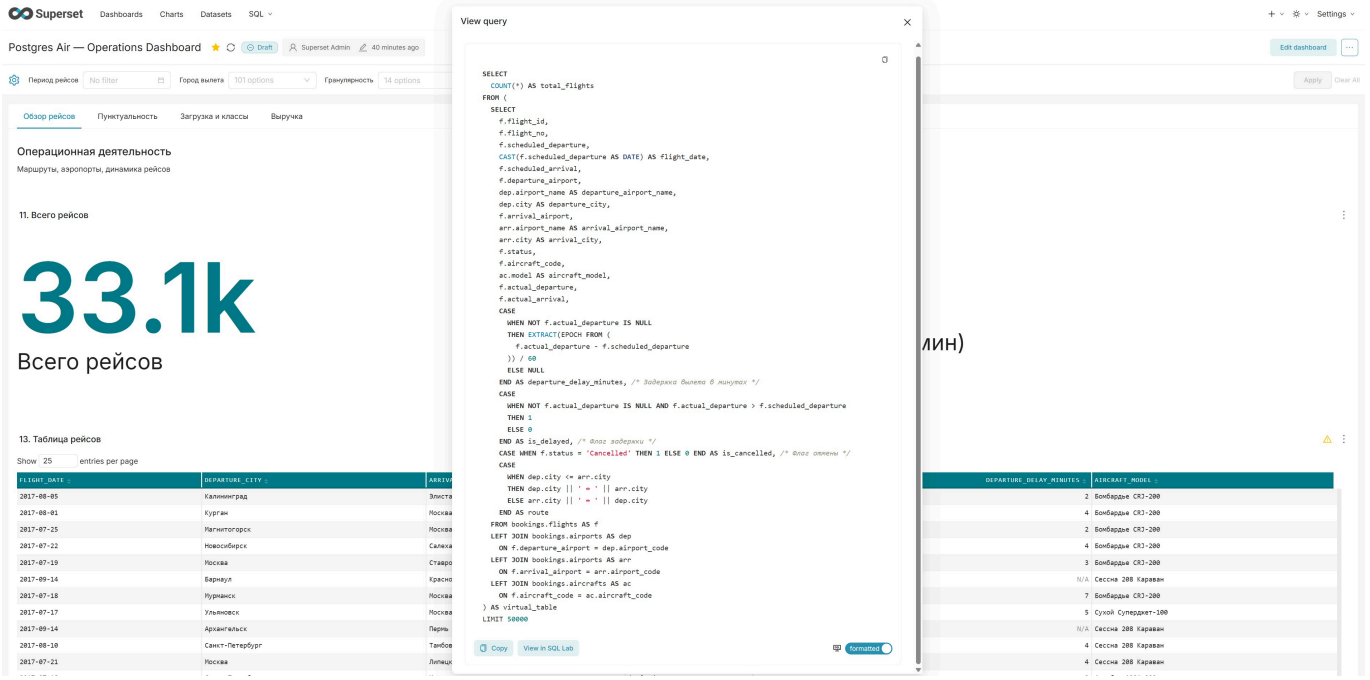
6) В данный фильтр добавили возможность множественного выбора, обязательное значение остается. Скорректируйте запрос, добавьте Jinja-шаблон с минимально необходимым функционалом.

```
{% if filter_values('departure_city') %}
WHERE dep.city = ANY({{ filter_values('departure_city') }})
{% endif %}
```



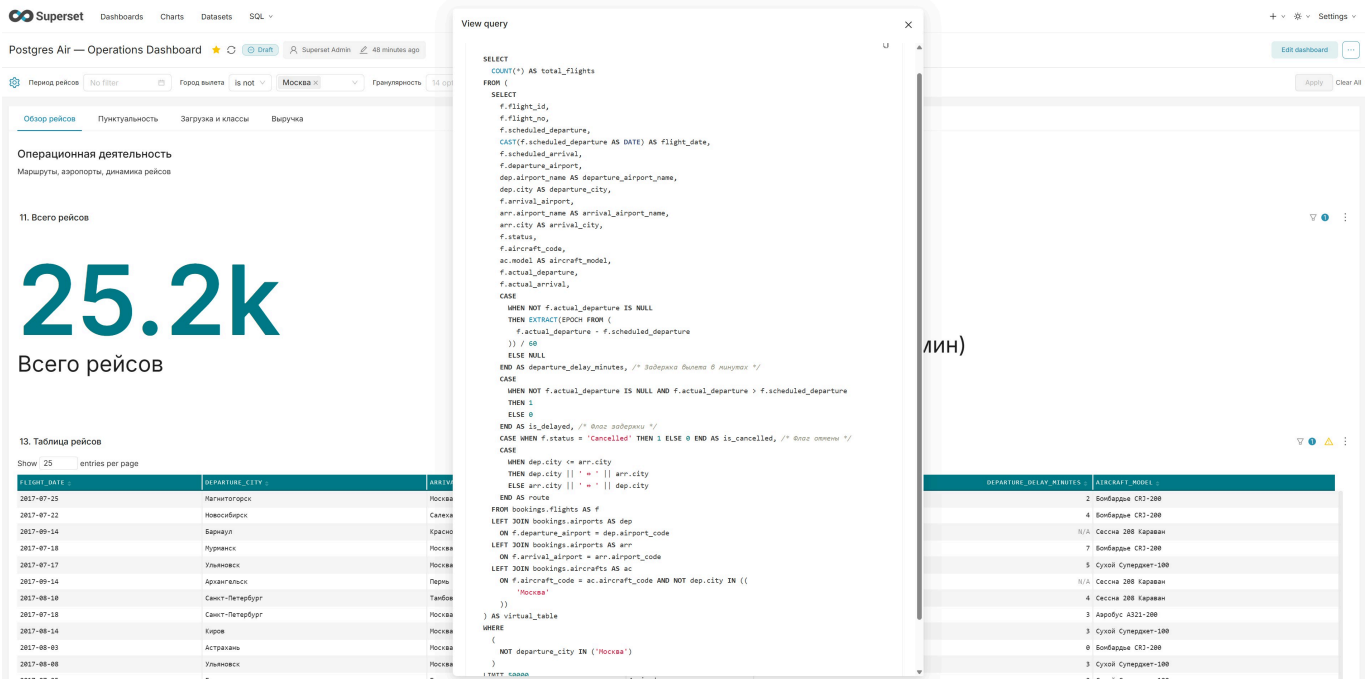
7) У данного фильтра убрали обязательное значение, защитите запрос от ошибок. Добавьте Jinja-шаблон с минимально необходимым функционалом.

```
{% if filter_values('departure_city') %}
WHERE dep.city = ANY({{ filter_values('departure_city') }})
{% endif %}
```



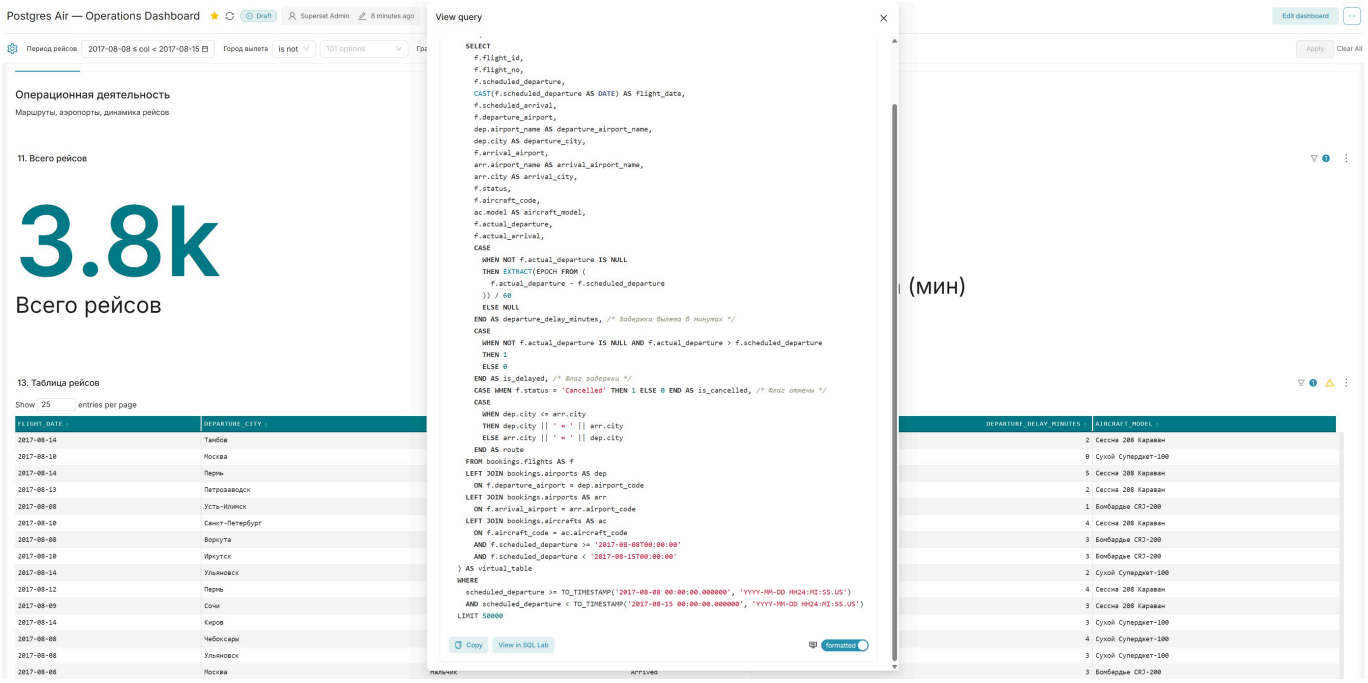
8) Учтите, что данный фильтр может, как выбирать значения, так и исключать. Добавьте Jinja-шаблон с минимально необходимым функционалом.

```
{% for filter in get_filters('departure_city') %}
  {% if filter.op == 'IN' %}
    AND dep.city IN ({{ filter.val | where_in }})
  {% elif filter.op == 'NOT IN' %}
    AND dep.city NOT IN ({{ filter.val | where_in }})
  {% endif %}
{% endfor %}
```



9) Добавьте в запрос фильтрацию по дате с помощью from_dttm/to_dttm.

```
{% if from_dttm is defined and from_dttm is not none %}
  AND f.scheduled_departure >= '{{ from_dttm }}'
{% endif %}
{% if to_dttm is defined and to_dttm is not none %}
  AND f.scheduled_departure < '{{ to_dttm }}'
{% endif %}
```



10) Организуйте фильтрацию по дате с помощью get_time_filter.

```
{% set tf = get_time_filter('f.scheduled_departure') %}
{% if tf.from_expr %}AND {{ tf.from_expr }}{% endif %}
{% if tf.to_expr %}AND {{ tf.to_expr }}{% endif %}
```

